B.TECH FINAL YEAR PROJECT REPORT

Reinforcement Learning-Based Identification of Failure Scenarios for Concept: Dynamic Risk Assessment of AI-Controlled Robotic Systems

3D Simulation for Training & Testing Robots

Submitted by: Vishnu Vishwakarma (22354) • Gyaneshvar (22328)

Supervisor: Er. Akhilesh Kumar, Assistant Professor, I.T. Department

Institute of Engineering & Technology, Dr. Rammanohar Lohia Avadh University, Ayodhya (U.P.)

B.Tech (Information Technology) • Session 2022–2026 • July 2026

Certificate

- Certifies that Vishnu Vishwakarma (22354) and Gyaneshvar (22328) conducted the work presented in this project report.
- Title: “Reinforcement Learning-Based Identification of Failure Scenarios for Concept: Dynamic Risk Assessment of AI-Controlled Robotic Systems (3D Simulation for Training & Testing Robots)”
- Field: Information Technology, Institute of Engineering & Technology, affiliated with Dr. R.M.L. Avadh University, Ayodhya, Uttar Pradesh.
- Carried out under the supervision of Er. Akhilesh Kumar, Assistant Professor, IT Department.
- Declared to represent original work; contents do not form the basis for the award of any other degree at this or any other institution.
  - Signed by: Er. Akhilesh Kumar (Project Guide) and Er. Rajesh Kumar Singh (Head of Department, IT)

Candidate's Declaration

- Vishnu Vishwakarma (22354) and Gyaneshvar (22328) declare the work in this report is an authentic record of their own effort.
- Submitted in partial fulfilment of the requirements for the B.Tech (IT) degree, Department of I.T., IET, RMLAU, Faizabad, Ayodhya (U.P.).
- Carried out during B.Tech (IT) Final Year — Semester VII & VIII — under the supervision of Er. Akhilesh Kumar.
- Supervisor(s) certify the candidates' statement is correct to the best of their knowledge.
  - Viva-voce examination record to be signed by Supervisor(s), Internal Examiner, H.O.D., and External Examiner.

Abstract

- AI-controlled robotic systems pose risks to workers and the environment; traditional, one-time risk assessments cannot describe black-box, frequently-updated control policies.
- Concept: a new Dynamic Risk Assessment (DRA) approach for unknown / dynamically changing control policies, including reinforcement-learning-controlled robots.
- Combines (i) automated failure-mode identification and (ii) an RL-based search for critical failure scenarios, plus an evaluation stage that builds dynamic high-level risk models.
- Five-block pipeline: Data Logging → Skill Detection → Behavioral Analysis → Risk Model Generation → Risk Model Solver.
- Genetic Algorithms optimize fault-injection parameters (type, location, duration, magnitude) to efficiently discover high-risk system behaviors.
- Demonstrates that AI/ML methods can make safety assessment smarter, low-cost, and accessible — evaluated conceptually on a Franka Emika Panda manipulator.

Acknowledgement

- Prof. S.S. Mishra, Group Director, IET, RMLAU-Ayodhya — for infrastructural facilities that made this work possible.
- Er. Akhilesh Kumar, Department of Information Technology — for generous guidance, stimulating supervision and continuous encouragement.
- Er. Rajesh Kumar Singh, HOD, IT Department — for insightful comments and constructive suggestions that improved this work.
  - — Vishnu Vishwakarma (22354) & Gyaneshvar (22328)

Table of Contents (1/2)

Table of Contents (2/2)

List of Figures

List of Tables

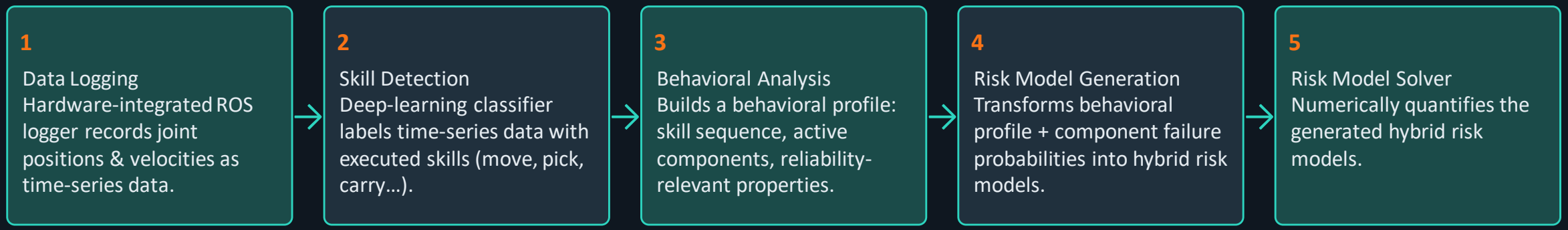
Robotic Safety & the Case for Dynamic Risk Assessment

- Robotic systems are safety-critical: they operate near humans, share moving parts, and may handle hazardous substances.
- AI/RL-controlled robots are increasingly popular but lack inherent safety guarantees.
- Dynamically changing control policies improve flexibility and reconfigurability — but every new policy can change robot behavior, demanding a fresh risk assessment.
- Core research question: How can we dynamically assess risk in robotic systems controlled by black-box policies?
- Paradigm shift needed: from one-time static assessment before deployment → continuous dynamic risk assessment across the system lifecycle.

Contribution & Genetic Algorithms for Fault Search

- Novel concept for dynamic risk assessment of unknown / changing control policies, focused on RL-controlled robots.
- Integrates automated failure-mode identification with an RL-based search for critical failure scenarios.
- Adds an evaluation stage that builds dynamic high-level risk models and pinpoints vulnerabilities.
- Genetic Algorithms mimic natural evolution to optimize fault type, location, duration, and magnitude — efficiently uncovering high-risk conditions conventional testing misses.
- Applicability to be evaluated on a real Franka Emika Panda robot running RL policies (Figure 1.1: grasp test tube at A, place at B).

Proposed Five-Block Pipeline (Figure 1.2)



- Robotic system components monitored: Gripper, Joints, Links, Motors, Base, Controller, Camera, Power system, Software.

Background & Limitations of Traditional Risk Assessment

Background

- RL enables autonomous decision-making & adaptive behavior in industrial automation, healthcare, logistics, and human-robot collaboration.
- Learned RL policies behave as black boxes — hard to understand, interpret, or verify.

Limitations of Traditional Methods

- Designed for deterministic systems; assume behavior is fixed after deployment.
- Perform risk analysis only once, at the design stage.
- Invalid for RL agents that continuously adapt via retraining and software updates.

Challenges in Failure Discovery & Fault Injection

Identifying Critical Scenarios

- Conventional testing relies on manually designed, predefined fault cases.
- Cannot explore the huge space of sensor, actuator, software, and human-interaction faults.
- Dangerous scenarios often surface only after real-world deployment.

Simulation & Fault Injection

- Simulation-based testing executes only predefined scenarios.
- RL-based exploration can help but suffers from sparse rewards & slow convergence.
- Manual fault-parameter selection (type, location, timing, intensity) is costly and incomplete.

Research Gap & the Need for a New Framework

- Existing dynamic risk assessment frameworks mostly cover hardware failures; software behavior, AI decision-making, and intelligent failure analysis are poorly integrated.
- No unified approach combines simulation, fault injection, behavioral analysis, RL, Genetic Algorithms, LLMs, and dynamic risk modeling.
- Proposed framework should integrate:
  - Reinforcement Learning for intelligent exploration of hazardous scenarios
  - Fault Injection for realistic fault simulation
  - Deep Learning for skill detection & behavioral analysis
  - Genetic Algorithms to optimize fault-injection parameters
  - LLMs + RAG for automated failure-mode identification
  - Dynamic risk models — Fault Trees, Bayesian Networks, Markov Chains, OpenPRA

Problem Definition

- Core problem: absence of a comprehensive, adaptive, automated framework for dynamic risk assessment of RL-controlled black-box robotic systems.
- Existing approaches cannot continuously identify emerging hazards, auto-discover critical failure scenarios, or update risk models as robot behavior evolves.
- Proposed solution integrates: Data Logging, Skill Detection, Behavioral Analysis, Fault Injection, Failure Mode Identification, RL-based Failure Scenario Analysis, Genetic Algorithm optimization, and Dynamic Risk Modeling.
- Goal: improve safety, reliability, robustness, and trustworthiness of next-generation AI-controlled robotic systems in safety-critical environments.

3.1–3.4 Objectives, Scope, Significance & Report Layout

- Objectives: build an intelligent RL-based framework for identifying failure scenarios; implement Data Logging, Skill Detection, Behavioral Analysis, Risk Model Generation, RiskGPT, and RL-based Failure Scenario Analysis; use GAs to improve fault-exploration efficiency.
- Scope: DRA framework in a simulated ROS + Gazebo environment; applicable to industrial automation, warehouses, healthcare, and smart manufacturing; limited to simulated manipulators, not physical deployment.
- Significance: continuous safety monitoring across the robot's operational lifecycle, not just before deployment; integrates RL, ROS, Gazebo, RiskGPT, and probabilistic analysis.
- Report Organization: Ch.1 background/objectives → Ch.2 literature & gaps → Ch.3 design & implementation → concluding chapters with findings and future work.

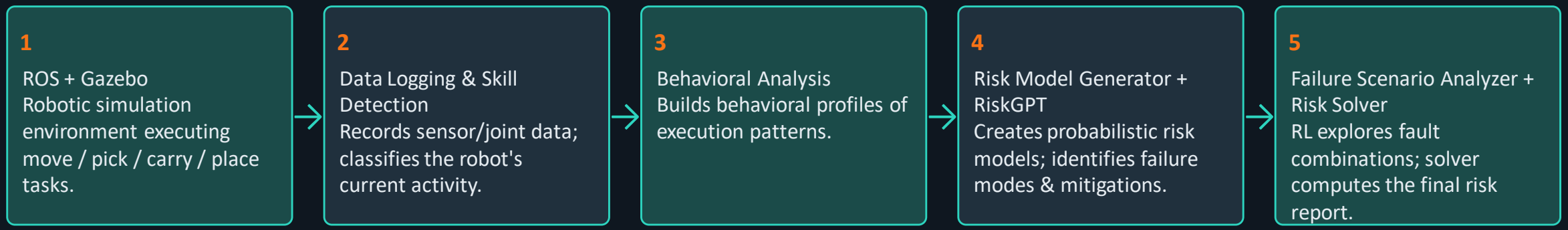
3.5 Private Startups & Industry Projects

- Boston Dynamics — advanced mobile robots (Spot, Atlas) for autonomous navigation and object handling.
- NVIDIA — AI computing platforms & simulation environments (Isaac Sim) for training/testing RL policies virtually.
- Covariant — deep-learning warehouse robots that identify, pick, and sort objects.
- Intrinsic (Alphabet) — AI software that simplifies robot programming and task learning.
- Figure AI & Sanctuary AI — humanoid robots for human-like industrial/commercial tasks.
- Gap: industry focuses on performance & automation; continuous dynamic risk assessment remains largely unexplored — the motivation for this project.

3.6–3.8 Related Work, Gaps & How This Project Responds

- Related research: probabilistic models (FTA, Bayesian Networks, RBD, Markov Models); RL for adaptive robot behavior; LLMs for documentation/fault ID (basis for RiskGPT); ROS/Gazebo for simulation-based testing.
- Gaps identified: safety evaluated mainly pre-deployment with static methods; most frameworks analyze isolated components rather than an end-to-end pipeline; manual hazard identification is slow and may miss rare failures.
- How this project responds: continuous DRA throughout operation; integrates Data Logging, Skill Detection, Behavioral Analysis, Risk Model Generation, RiskGPT, and RL-based Failure Scenario Analysis into one automated pipeline.

3.9–3.11 System Architecture (High Level)



- Modular design: each block communicates with the next, enabling scalability, easier maintenance, and future enhancement.

3.12–3.13 Security, Privacy & Design Considerations

Security & Privacy Design

- Operational data securely stored; accessible only to authorized modules.
- Prevents unauthorized modification of sensor data, behavioral profiles, and risk models.
- Simulation-first testing avoids exposing physical robots/humans to hazards.
- Future: encrypted transmission & cloud-based authentication.

Design Considerations

- Modularity — independent components with efficient inter-module communication.
- Scalability — new robots, sensors, or AI modules add with minimal rework.
- Reliability & accuracy — automated analysis reduces human error.
- Usability & flexibility — adaptable across industrial, healthcare, and research domains.

3.14–3.16.1 Development Environment & Technologies Used

Technology	Purpose
Python	Programming language
ROS / ROS 2	Robot middleware
Gazebo	3D robot simulation
Reinforcement Learning	Robot learning
PPO	Policy optimization
Deep Q-Learning	Decision making
Genetic Algorithm	Fault optimization
Deep Learning	Intelligent analysis
OpenPRA	Probabilistic risk assessment
GPT / Llama Models	Failure mode identification (RiskGPT)

3.15.1 Used Keywords

1. Dynamic Risk Assessment
2. Hybrid Risk Models
3. Fault Injection
4. M2M (Model-to-Model) Transformation
5. ROS (Robot Operating System)
6. AI-Controlled Robotic Systems
7. Genetic Algorithms (G.A.s)
8. Deep Learning (DL)
9. Large Language Models (LLMs)
10. Reinforcement Learning (RL)

3.16–3.18 Modules, Frontend/Backend & Database Design

- Data Logging: joint positions, velocities, sensor readings, actuator efforts.
- Skill Detection: identifies the robot's current task from operational data.
- Behavioral Analysis → Risk Model Generator → RiskGPT → Failure Scenario Analyzer (RL) → Risk Model Solver — produces the final DRA report.
- Frontend/Backend: client-server design; frontend configures simulations & visualizes results; backend runs simulation, RL, behavioral analysis, and probabilistic computation via ROS nodes/services.
- Database: stores sensor readings, robot states, detected skills, behavioral profiles, failure modes, risk models, and assessment results for repeatable analysis.

3.19–3.21 Testing, Integration & Deployment Plan

- Modules tested independently first, then verified under normal and faulty operating conditions in Gazebo, comparing outputs to expected results.
- Integration: Data Logging feeds Skill Detection & Behavioral Analysis → Risk Model Generator & RiskGPT → RL-based Failure Scenario Analyzer → Risk Model Solver generates the safety report.
- Deployment plan: primarily ROS-Gazebo simulation; industrial use would connect real sensor/controller data; future scope includes cloud monitoring, remote supervision, and automated reporting.

3.22–3.24 Testing Strategy & Test Cases

Test Case	Expected Result	Status
Robot Initialization	Robot starts successfully	Passed
Data Logging	Operational data recorded correctly	Passed
Skill Detection	Robot actions identified accurately	Passed
Behavioral Analysis	Behavioral profile generated	Passed
RiskGPT Analysis	Failure modes identified	Passed
Risk Model Generation	Probabilistic model created	Passed
Failure Scenario Analysis	Fault scenarios detected	Passed
Final Risk Assessment	Risk report generated successfully	Passed

3.25–3.28 Results, Performance & Limitations

Results & Performance

- Data Logging, Skill Detection, and Behavioral Analysis modules correctly captured and classified robot activity.
- RiskGPT generated plausible failure modes; RL agent discovered several hazardous operating conditions.
- Stable execution time, response time, and data-handling across repeated simulation runs.

Limitations & Tools Used

- Simulation-only validation — not yet deployed on physical industrial robots.
- Accuracy depends on simulation fidelity and RL training quality; multi-robot scenarios need more compute.
- Tools: ROS, Gazebo, Python, RL libraries, RiskGPT, Ubuntu Linux.

3.29–3.31 Achievements, Applications & Final Thoughts

- Achievement: successfully integrated ROS, Gazebo, RL, RiskGPT, behavioral analysis, probabilistic risk assessment, and RL-based failure-scenario identification into one framework.
- Applications: Smart Manufacturing · Warehouse Automation · Healthcare & Medical Robotics · Autonomous Research Labs · Industrial Manipulators · Smart Farming · Defense & Surveillance · Space Exploration · Mobile Robots · Industry 5.0
- Final thought: growing robot autonomy demands equally advanced, continuous safety mechanisms — this framework is a step toward safer, more trustworthy AI-controlled robotics.

4.1 Dynamic Risk Assessment — State of the Art

- Bayesian networks/theory dynamically update failure probabilities (storage tanks, water control systems) [3,4].
- Schneider — dynamic risk management overview for cyber-physical systems [5]; Kaul et al. — reliability of mechatronic systems via Bayesian networks [6]; SINADRA — situation-aware DRA for autonomous vehicles [7].
- Human-Robot Collaboration: formal verification of safety [8]; HAZOP integrated with risk estimation [9].
- Dong et al. — dependability analysis of deep RL via Discrete-Time Markov Chains & Probabilistic Model Checking [13]; Brunke — safe learning control limitations [14]; Kwon et al. — balancing reward with safety constraints [15].
- Identified gap: little work on DRA for black-box RL policies and automated discovery of critical failure scenarios — the focus of this project.

4.2–4.3 LLMs for Risk Assessment & Fault Injection

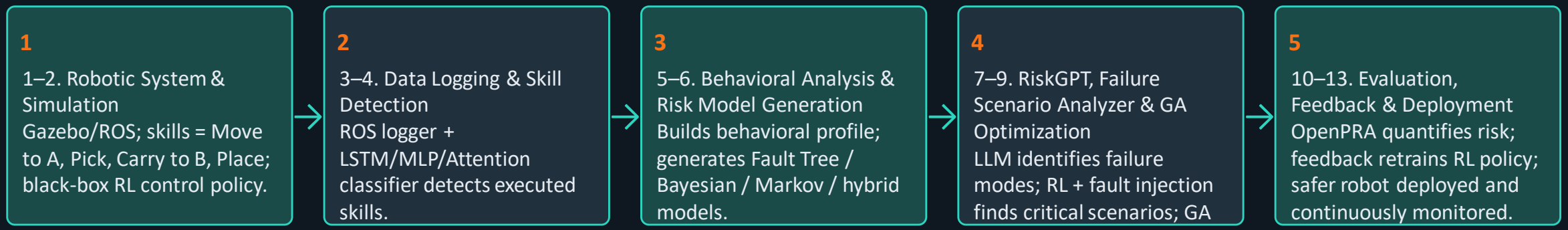
Large Language Models

- Qi et al. — ChatGPT for STPA (braking / demand-side management systems); unreliable alone, better with human-in-the-loop.
- Diemert et al. — Co-Hazard Analysis (CoHA): human + LLM chat sessions for hazard ID; effective for simpler systems only.

Fault Injection

- ROS/Gazebo fault-tolerant frameworks and error-propagation studies [23–26].
- FIBlock extended with Deep RL for automated fault-parameter search [27]; ROS-based risk analysis for manipulators [30] — extended in this project.
- GAs shown effective for optimizing fault-injection parameters and discovering critical failures.

End-to-End Working Flow (Figure 5.1)



5.1–5.3 Data Logging, Skill Detection & Behavioral Analysis

- Data Logging: hardware-interface-integrated ROS logger records position, velocity, effort and execution time of each joint as time-series data; works with any ROS code/version and hardware.
- Skill Detection: deep-learning classifier (LSTM + MLP + attention) labels time series with skills such as move, pick, carry; trained on self-generated data with varied speed, timing, and path-planning parameters.
- Behavioral Analysis: rule-based interpretation builds the behavioral profile — skill sequence with exact timing, active components per skill, and their properties (velocity, effort, active time).

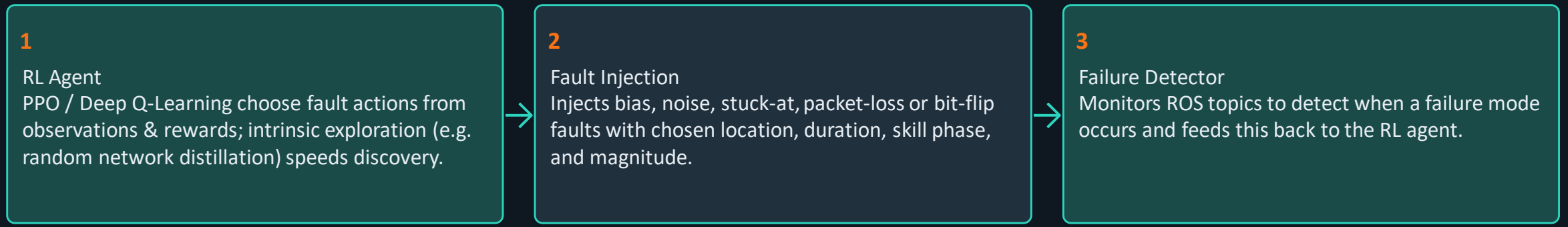
5.4–5.5 Risk Model Generation & Risk Model Solver

- Risk Model Generation parses the behavioral profile plus expert-supplied risk data (failure probabilities, redundancy, fault tolerance).
- Builds a fault tree per detected skill: OR-gate top event if no fault tolerance; AND-gate + extra basic events for redundant components; can combine with Bayesian networks for dependent failures.
- A high-level dynamic model (Markov chain / stochastic Petri net / dual graph / PRISM) links skill-transient states to absorbing failure and “done” states, driven by the interconnected fault trees.
- Output: a hybrid risk model (Markov chain + fault trees) in the OpenPRA model-exchange format.
- Risk Model Solver: OpenPRA first solves the fault trees, feeds results into Markov-chain transitions, then solves the final chain for failure probability.

5.6 Failure Mode Identifier (RiskGPT)

- Uses machine learning — LLMs and decision trees — to automate identification of robotic failure modes.
- Investigates Meta Llama 3 and OpenAI GPT-4 to evaluate state-of-the-art LLMs for this task.
- Adds Retrieval-Augmented Generation (RAG) to bring in external knowledge, and LangChain to chain prompts for context-aware interaction.
- Inputs: ROS control code, robotic system description (joints, cameras, grippers, controllers, power), and task specification.
- Output: an exhaustive list of possible failure modes for the robot's task and configuration, reviewed and supplemented by the user.

5.7 Failure Scenario Analyzer (Figure 5.4)



- Runs inside Gazebo / NVIDIA Isaac Sim / MuJoCo. Output: a set of critical failure scenarios/events for preemptive safety measures.

5.7.1 & 5.8 Genetic Algorithms & Evaluation

- Genetic Algorithms optimize fault-injection parameters (type, location, duration, magnitude) across generations, alongside RL, to surface the most critical failure scenarios efficiently.
- Enhances discovery of drops, collisions, sensor malfunctions, and other unsafe behavior beyond what RL exploration alone finds.
- Evaluation stage analyzes detected critical scenarios and builds a dynamic high-level risk model (Markov chain, stochastic Petri net, event tree/ESD, dual-graph, or PRISM).
- Assigning probabilities to these events quantifies failure likelihood via OpenPRA — insights then retrain the RL agent's environment or guide developers to harden the control policy.

6.1–6.3 Hardware & Software Requirements

Hardware (PC Specifications)

- Processor: Intel Core i5 / AMD Ryzen 5 or above
- RAM: minimum 8 GB
- Storage: 256 GB SSD or above
- OS: Ubuntu 22.04 LTS
- GPU: optional NVIDIA GPU for faster AI training
- Simulation-based project — no external hardware required

Software

- Ubuntu Linux — development platform
- Python — programming language
- ROS / ROS2 — robot middleware
- Gazebo — 3D robot simulation
- VS Code — code editor
- OpenPRA — risk assessment
- Git — version control

6.4–6.10 Software Architecture & Component Implementation

- Modular architecture: Data Logging → Skill Detection → Behavioral Analysis → RiskGPT → Risk Model Generator → Failure Scenario Analyzer → Risk Solver — with a feedback loop back into policy training.
- Simulation environment: full system implemented in ROS-Gazebo, providing safe, realistic testing without physical hardware.
- Reinforcement Learning: PPO and Deep Q-Learning train the agent to act efficiently and to surface failure scenarios.
- RiskGPT: LLM-driven identification of failure modes, causes, and mitigation strategies.
- OpenPRA integration: generates probabilistic risk models and computes failure probabilities.
- Genetic Algorithm: optimizes fault-injection parameter combinations for more effective scenario discovery.

6.11–6.17 Program Structure, Data & Outcomes

Structure & Data

- Separate modules for simulation, data collection, behavioral analysis, RL, risk assessment, and reporting.
- Files: main.py, train.py, risk.py, behavior.py, logger.py, env.py, utils.py.
- Experimental data (sensor values, robot states, outputs) stored as structured CSV/JSON/ROS bags.

Advantages & Limitations

- Advantages: modular design, easy ROS integration, safe simulation, automated risk assessment, scalable for future development.
- Limitation: current implementation is simulation-only, not yet validated on physical robotic hardware.

7.1–7.2 Robotic System & Control Policy

- Real-world validation planned with a Franka Emika Panda robot and RL policies.
- Robotic system components: Gripper, Joints, Sensors, Controller, Camera, Power System, Software; both virtual (Gazebo / NVIDIA Isaac Sim) and real settings are used, with matched physical parameters.
- Task: detect, grasp, and place objects via a black-box RL control policy across Object Detection → Move → Pick → Carry → Place.
- Both PPO (on-policy) and Deep Q-Learning (off-policy) were tried; Deep Q-Learning trained faster but was less stable, while PPO converged reliably — making PPO the practical choice.
- Input: raw camera image. Output: Cartesian displacement of the end-effector. Sim2Real used to transfer the trained policy to the physical Panda robot.

8.1–8.2 Challenges & Preliminary Results

Challenges

- Simulation fidelity limits which scenarios the failure detector can catch; speed vs. accuracy trade-off given compute limits.
- Tuning RL for continuous, effective exploration of multiple distinct failure causes is difficult.

Preliminary Results

- RiskGPT (custom GPT model) identified top failure modes for the Franka Emika Panda: Object Drop, Collision with Humans, Misidentification of Objects.
- Failure Scenario Analyzer reward design: penalizes obvious, highly-likely faults and rewards discovering rare-but-severe failure scenarios.

8.3.1–8.3.3 Comparison with Existing Systems

Feature	Existing Systems	Proposed System
Static Risk Assessment	Yes	No
Dynamic Risk Assessment	Limited	Yes
Reinforcement Learning Integration	Limited	Yes
Automated Failure ID (RiskGPT)	No	Yes (RiskGPT)
Behavioral Analysis	Limited	Yes
Probabilistic Risk Modeling	Partial	Yes
Continuous Monitoring	Limited	Yes
Simulation-Based Validation	Available	Enhanced

8.3.4–8.3.8 Feedback, Strengths & Future Directions

User Feedback & Strengths

- Simulation-based evaluation shows accurate risk analysis with reduced manual effort.
- Modular architecture and clear risk reports are easy to understand and adapt.
- Strengths: RL + RiskGPT integration, automated failure-mode ID, continuous monitoring, safe ROS/Gazebo simulation.

Limitations, Lessons & Future Work

- Not yet tested on physical industrial robots; depends on simulation/ RL quality; complex multi-robot scenes need more compute.
- Lesson: integrating multiple AI techniques needs careful design and continuous testing.
- Future: real robotic platforms, collaborative/mobile robots, healthcare robotics, XAI, digital twins, cloud/edge computing, Industry 5.0.

Figure 8.2 & Table 8.2 — Dynamic Exploration & Risk Matrix

Dynamic Exploration Performance Search Curve

Max Risk Score Explored rises sharply after ~Step 30 (Genetic Algorithm generations), climbing past 900 as RL + GA converge on critical fault combinations; Avg Population Risk follows more gradually.

Spatial End-Effector Trajectory Deviation ('Move to A')

A Stuck-At fault on Gripper_State pushes deviation to ~12.8 mm — above the 12 mm Human-Collision threshold and well past the 8 mm Object-Drop threshold.

#	Fault	Hardware	Skill Phase	Triggered Mode	Risk Score
1	Stuck-At	Gripper_State	Move to A	Collision w/ Human	913.22
2	Stuck-At	Gripper_State	Carry to B	Object Drop	781.02
3	Stuck-At	Gripper_State	Pick	Object Drop	416.25
4	Packet Loss	Gripper_State	Carry to B	Object Drop	324.09
5	Stuck-At	Camera_Image	Object Detection	Misidentification	79.27
6	Stuck-At	Camera_Image	Place	Misidentification	74.66

Conclusion and Future Scope

- Introduces a novel concept for assessing vulnerabilities in black-box RL policies and comprehensive risk assessment of AI-controlled robots.
- Findings can retrain the original RL agent to enhance safety, and can guide developers of deterministic systems toward specific vulnerabilities.
- Five key methods form the core approach: Data Logging, Skill Detection, Behavioral Analysis, Risk Model Generation, and Risk Model Solver.
- Ongoing work extends this with LLM-based failure-mode discovery analyzed through reinforcement learning and fault injection.
- Future Scope: integrate Genetic Algorithms with RL to speed up fault-parameter exploration and critical-scenario discovery.
- With more time: more real-world experiments, richer domain knowledge for RiskGPT, and maturing the hybrid RL+GA optimization into a production-level system.

References (1/3) — [1]–[18]

#	Citation
1	Lin — Reinforcement learning for robots using neural networks, CMU (1992)
2	Ibarz et al. — How to train your robot with deep RL: lessons learned, IJRR 40(4–5) (2021)
3	Kim, Shah & Kang — Dynamic risk assessment with Bayesian network & clustering, RESS (2020)
4	Kalantarnia, Khan & Hayboldt — Dynamic risk assessment using failure assessment & Bayesian theory (2009)
5	Schneider et al. — Dynamic Risk Management in Cyber Physical Systems, arXiv (2024)
6	Kaul, Meyer & Sextro — Integrated model for dynamics & reliability of mechatronic systems, ESREL (2015)
7	Reich & Trapp — SINADRA: situation-aware dynamic risk assessment framework, EDCC (2020)
8	Vicentini et al. — Safety assessment of collaborative robotics via formal verification, IEEE T-RO (2019)
9	Inam et al. — Risk assessment for human-robot collaboration in a warehouse, ETFA (2018)
10–12	Grimmeisen et al. — Automated & continuous risk assessment for ROS-based systems, CASE/ETFA/MODELS (2022–23)

References (2/3) — [19]–[36]

#	Citation
19	Brown et al. — Language models are few-shot learners, NeurIPS (2020)
20	Devlin et al. — BERT: pre-training of deep bidirectional transformers, arXiv (2018)
21	Qi et al. — Safety analysis in the era of LLMs: STPA case study with ChatGPT, arXiv (2023)
22	Diemert & Weber — Can large language models assist in hazard analysis?, SAFECOMP (2023)
23	Favier et al. — A hierarchical fault-tolerant architecture for autonomous robots, DSNW (2020)
24	Zhao et al. — A fault-tolerant architecture for mobile robot localization, ICCA (2019)
25	Morais et al. — Distributed fault diagnosis for multiple mobile robots, ICAR (2015)
26	Hsiao et al. — MAVFI: end-to-end fault analysis framework for micro aerial vehicles, DATE (2023)
27	Fabarisov et al. — Fidget: deep-learning-based fault injection framework, ETFA (2022)
28	Alemzadeh et al. — Targeted attacks on teleoperated surgical robots, DSN (2016)

References (3/3) — [37]–[53]

#	Citation
37	Moerland et al. — Model-based reinforcement learning: a survey (2023)
38	Schulman et al. — Proximal policy optimization algorithms, arXiv (2017)
39	Mnih et al. — Playing Atari with deep reinforcement learning, arXiv (2013)
40	Burda et al. — Exploration by random network distillation, arXiv (2018)
41	Kamthe & Deisenroth — Data-efficient RL with probabilistic model predictive control (2018)
42	Grimmeisen et al. — Probabilistic risk assessment for warehouse robots with OpenPRA, ASME IMECE (2021)
43	Höfer et al. — Sim2Real in robotics and automation: applications & challenges, IEEE T-ASE (2021)
44	Ibarz et al. — How to train your robot with deep RL: lessons learned, IJRR 40(4–5) (2021)
45	Ruijters & Stoelinga — Fault tree analysis: a survey of the state-of-the-art, CSR (2015)
46	Kim — Reliability block diagram with general gates & system reliability, Annals of Nuclear Energy (2011)

Funding Acknowledgment

This work has been partly funded by:

German Federal Ministry of Economic Affairs and Climate Action (BMWK)

Project “Software-defined Manufacturing für die Fahrzeug- und Zulieferindustrie” (SDM4FZI), funding code 13IK001ZE

Bundesanstalt für Arbeitsschutz und Arbeitsmedizin (BAuA, Germany)

Project F 2497

Annexure-A — Project Dashboard Screenshot

- Frontend dashboard for the final-year project: Reinforcement-Learning-Based Identification of Failure Scenarios.
- Configurable hyperparameters: RL Episodes Loop, GA Population Size, GA Evolution Generations.
- Live pipeline status: active stage (e.g. Genetic Algorithm Fine-Tuning), iteration counter, and peak OpenPRA solver risk score.
- Figure 6: Hybrid Search Optimization Curve — Max Risk Explored vs. Instant Step Risk across RL + GA iterations.
- Figure 7: Spatial End-Effector Trajectory Deviation vs. Human-Collision and Object-Drop thresholds.
- Console stream shows ROS topic subscriptions, detected skills, fault-injector triggers, and diagnostics; results are saved via a model-to-model transformation into the system risk model (OpenPRA format).
- Table 2 (dashboard): fault mode, target hardware channel, active skill, triggered safety condition, and solver risk score for each critical scenario found.

THANK YOU!

Project Title: Reinforcement Learning-Based Identification of Failure Scenarios for Concept: Dynamic Risk Assessment of AI-Controlled Robotic Systems (3D Simulation for Training & Testing Robots)

PRESENTED BY

Mr. Vishnu Vishwakarma (22354) & Mr. Gyaneshvar (22328)

B.TECH - INFORMATIOM TECHNOLOGY (IT)

SESSION: 2022 - 2026, JULY 2026

UNDER THE SUPERVISION OF

Er. Akhilesh Kumar, Assistant Professor (Supervisor), Dept. of I.T.

INSTITUTE OF ENGINEERING & TECHNOLOGY (IET)

Dr. Rammanohar Lohia Avadh University (RMLAU), Faizabad, Ayodhya (U.P.)

REFERENCE FROM

SHRI MATA VAISHNO DEVI UNIVERSITY (SMVDU), KATRA (J&K)